Solar Power Generation Forecasting

MLOps Project — Complete Documentation

CI/CD Pipeline + AWS Deployment + CloudWatch Monitoring

Personal Reference Guide

April 2026

Complete Tech Stack

Tool	Purpose	Where Used
DVC	Data versioning	Local machine
MLflow	Experiment tracking	Local machine
Scikit-learn	ML model training	Local machine
Streamlit	Web UI for predictions	Docker container
FastAPI	REST API for predictions	Docker container
Docker	Containerization	Local + CI/CD + EC2
GitHub Actions	CI/CD pipeline	GitHub cloud
AWS ECR	Docker image registry	AWS cloud
AWS EC2	App hosting server	AWS cloud
CloudWatch	Logging & monitoring	AWS cloud

1. Project Overview

This project implements an end-to-end MLOps pipeline for solar power generation forecasting. The system predicts AC power output based on solar irradiation, ambient temperature, and time of day.

Architecture

```
Local Machine
    │  git push
    ▼
GitHub Actions (CI/CD)
    │
    ├── Install dependencies
    ├── Build Docker image
    ├── Push to AWS ECR
    └── Deploy to AWS EC2
            │
            ├── :8501  Streamlit (UI)
            └── :8000  FastAPI (API)
```

2. Project Folder Structure

```
Solar_Power_Generation_Forecasting/
├── .github/
│   └── workflows/
│       └── ci-cd.yml          ← GitHub Actions pipeline
├── data/
│   ├── raw/                   ← DVC tracked raw CSV files
│   └── processed/train.csv
├── app/
│   ├── app.py                 ← Streamlit UI
│   ├── api.py                 ← FastAPI REST API
│   ├── config.py              ← configuration
│   ├── model_loader.py        ← loads model.pkl
│   ├── utils.py               ← feature engineering + CSV helpers
│   ├── weather_service.py     ← OpenWeatherMap integration
│   ├── Dockerfile             ← container definition
│   ├── start.sh               ← launches FastAPI + Streamlit
│   └── model/
│       └── model.pkl          ← trained ML model
├── src/
│   ├── preprocess.py
│   ├── train.py
│   ├── registry.py
```

```
│   └── export_model.py
├── tests/
│   └── test_pipeline.py
└── requirements.txt
```

3. Running Locally

3.1 Setup Virtual Environment

```
# Create venv
python -m venv .solarprojvenv

# Activate (Windows)
.solarprojvenv\Scripts\activate

# Install dependencies
pip install -r requirements.txt
```

3.2 Run Streamlit App

```
cd app
streamlit run app.py
```

Opens at: http://localhost:8501

3.3 Run FastAPI

```
cd app
uvicorn api:app --host 0.0.0.0 --port 8000 --reload
```

Opens at: http://localhost:8000/docs (Swagger UI)

3.4 Run Both Together (Two Terminals)

Terminal 1	Terminal 2
cd app	cd app
streamlit run app.py	uvicorn api:app --port 8000 --reload
http://localhost:8501	http://localhost:8000/docs

3.5 Set Weather API Key

```
# Windows
set WEATHER_API_KEY=your_key_here

# Then run streamlit
streamlit run app.py
```

4. FastAPI — REST API

4.1 What is FastAPI?

FastAPI exposes your ML model as an HTTP API. Unlike Streamlit (for humans), FastAPI is for machines — other apps, scripts, or services can get predictions programmatically.

4.2 Endpoints

Method	Endpoint	Description
GET	/health	Check if model is loaded
POST	/predict	Manual prediction (send datetime + irradiation + temp)
POST	/predict/now	Auto prediction using current time + live weather
GET	/weather	Fetch live weather (needs API key)
GET	/history?limit=10	Last N rows from predictions.csv

4.3 Example Prediction Request (PowerShell)

```
Invoke-RestMethod \
  -Uri "http://localhost:8000/predict" \
  -Method POST \
  -ContentType "application/json" \
  -Body '{
    "datetime_str": "2024-06-15 13:30:00",
    "irradiation": 0.75,
    "amb_temp": 32.5,
    "save": true
  }'
```

4.4 Response

```
{
  "ac_power_kw": 3240.50,
  "capacity_pct": 64.81,
```

```
  "hour": 13,
  "saved": true
}
```

5. Docker

5.1 Dockerfile (app/Dockerfile)

```
FROM python:3.10-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
EXPOSE 8501
CMD ["streamlit", "run", "app.py",
     "--server.port=8501", "--server.address=0.0.0.0"]
```

5.2 Build Image Locally

```
docker build -t solar-app ./app
```

5.3 Run Container Locally

```
docker run -d -p 8501:8501 -p 8000:8000 \
  -e WEATHER_API_KEY=your_key \
  --name solar-container \
  solar-app
```

5.4 Useful Docker Commands

Command	Description
docker ps	List running containers
docker logs solar-container	View container logs
docker stop solar-container	Stop container
docker rm solar-container	Remove container
docker images	List all images
docker system prune -a -f	Delete all unused images (free disk space)

6. CI/CD Pipeline (GitHub Actions)

6.1 Pipeline Flow

```
git push
    |
```

```
    ▼
GitHub Actions (.github/workflows/ci-cd.yml)
    │
    ├── 1. Checkout code
    ├── 2. Setup Python 3.10
    ├── 3. Install dependencies
    ├── 4. Copy requirements to app/
    ├── 5. Build Docker image
    ├── 6. Run container (test)
    ├── 7. Login to AWS ECR
    ├── 8. Push image to ECR
    └── 9. SSH into EC2 → pull → run
```

6.2 Complete ci-cd.yml

```yaml
name: CI-CD Pipeline

on:
  push:
    branches:
      - main

jobs:
  build-test-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.10"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install dvc mlflow pytest

      - name: Copy requirements for Docker
        run: |
          cp requirements.txt app/requirements.txt

      - name: Build Docker image
        run: |
          docker build -t solar-app ./app
```

```
      - name: Run Docker container
        run: |
          docker run -d -p 8000:8000 \
            -e WEATHER_API_KEY=${{ secrets.WEATHER_API_KEY }} \
            --name solar-container \
            solar-app

      - name: Login to ECR
        run: |
          aws ecr get-login-password --region ${{ secrets.AWS_REGION }} | \
          docker login --username AWS \
          --password-stdin ${{ secrets.ECR_REGISTRY }}
        env:
          AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
          AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}

      - name: Push to ECR
        run: |
          docker tag solar-app:latest \
            ${{ secrets.ECR_REGISTRY }}/mlops/solar-power-forecasting:latest
          docker push \
            ${{ secrets.ECR_REGISTRY }}/mlops/solar-power-forecasting:latest

      - name: Deploy to EC2
        uses: appleboy/ssh-action@v1.0.0
        with:
          host: ${{ secrets.EC2_HOST }}
          username: ubuntu
          key: ${{ secrets.EC2_SSH_KEY }}
          script: |
            aws ecr get-login-password --region ${{ secrets.AWS_REGION }} | \
            docker login --username AWS \
            --password-stdin ${{ secrets.ECR_REGISTRY }}
            docker stop solar-container || true
            docker rm solar-container   || true
            docker system prune -a -f
            docker pull ${{ secrets.ECR_REGISTRY }}/mlops/solar-power-
forecasting:latest
            docker run -d -p 8000:8000 -p 8501:8501 \
              -e WEATHER_API_KEY=${{ secrets.WEATHER_API_KEY }} \
              --name solar-container \
              --restart always \
              --log-driver=awslogs \
              --log-opt awslogs-region=eu-north-1 \
              --log-opt awslogs-group=/solar-app/container \
              --log-opt awslogs-stream=solar-container \
              ${{ secrets.ECR_REGISTRY }}/mlops/solar-power-forecasting:latest
```

6.3 GitHub Secrets Required

Secret Name	Value
AWS_ACCESS_KEY_ID	Your AWS access key
AWS_SECRET_ACCESS_KEY	Your AWS secret key
AWS_REGION	eu-north-1
ECR_REGISTRY	435998720725.dkr.ecr.eu-north-1.amazonaws.com
EC2_HOST	EC2 public IP (update each time EC2 starts)
EC2_SSH_KEY	Full contents of solar-key.pem file
WEATHER_API_KEY	Your OpenWeatherMap API key

⚠ *EC2_HOST changes every time you stop/start EC2. Always update this secret before pushing.*

7. AWS Setup

7.1 AWS CLI Setup (Local Machine)

```
# Install AWS CLI
# Download from: https://aws.amazon.com/cli/

# Configure
aws configure
  AWS Access Key ID:     AKIAWLA4MQLKZKTQ37WE
  AWS Secret Access Key: your_secret_key
  Default region:        eu-north-1
  Default output format: json
```

7.2 AWS ECR — Container Registry

Create Repository

```
aws ecr create-repository --repository-name mlops/solar-power-forecasting --
region eu-north-1
```

Login to ECR

```
aws ecr get-login-password --region eu-north-1 | \
docker login --username AWS --password-stdin \
435998720725.dkr.ecr.eu-north-1.amazonaws.com
```

Tag and Push Image

```
docker tag solar-app:latest \
  435998720725.dkr.ecr.eu-north-1.amazonaws.com/mlops/solar-power-
forecasting:latest


docker push \
  435998720725.dkr.ecr.eu-north-1.amazonaws.com/mlops/solar-power-
forecasting:latest
```

ECR Lifecycle Policy (Auto-delete old images)

AWS Console → ECR → mlops/solar-power-forecasting → Lifecycle policy → Create rule:

- Rule priority: 1
- Image status: Untagged
- Match criteria: Image count more than 1
- Action: Expire

7.3 AWS EC2 — App Server

EC2 Details

Setting	Value
Instance ID	i-0dab48287a4d5d0dc
Instance Name	solar-app-server
Instance Type	t3.micro
OS	Ubuntu 22.04 LTS
Region	eu-north-1 (Stockholm)
Key Pair	solar-key.pem
Security Group	launch-wizard-2

Security Group Rules

Port	Protocol	Source	Purpose
22	TCP	0.0.0.0/0	SSH access
8000	TCP	0.0.0.0/0	FastAPI
8501	TCP	0.0.0.0/0	Streamlit

SSH into EC2

```
ssh -i "C:\Users\Ritesh gawade\Downloads\solar-key.pem" ubuntu@EC2_PUBLIC_IP
```

EC2 Initial Setup (Run Once)

```
# Install Docker
sudo apt update
sudo apt install docker.io -y
sudo usermod -aG docker ubuntu
newgrp docker

# Install AWS CLI
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o
"awscliv2.zip"
sudo apt install unzip -y
unzip awscliv2.zip
sudo ./aws/install

# Configure AWS
aws configure
```

Deploy on EC2 (Manual)

```
# Login to ECR
aws ecr get-login-password --region eu-north-1 | \
docker login --username AWS --password-stdin \
435998720725.dkr.ecr.eu-north-1.amazonaws.com

# Pull image
docker pull 435998720725.dkr.ecr.eu-north-1.amazonaws.com/mlops/solar-power-
forecasting:latest

# Run container
docker run -d -p 8000:8000 -p 8501:8501 \
  -e WEATHER_API_KEY=your_key \
  --name solar-container \
  --restart always \
  435998720725.dkr.ecr.eu-north-1.amazonaws.com/mlops/solar-power-
forecasting:latest
```

8. CloudWatch Logging

8.1 Setup

Step 1 — Create Log Group

AWS Console → CloudWatch → Log groups → Create log group

- Name: /solar-app/container

Step 2 — Create IAM Role

AWS Console → IAM → Roles → Create role:

- Trusted entity: EC2
- Permission: CloudWatchLogsFullAccess
- Name: EC2-CloudWatch-Role

Step 3 — Attach Role to EC2

EC2 → your instance → Actions → Security → Modify IAM role → EC2-CloudWatch-Role

Step 4 — Enable in Docker Run (in ci-cd.yml)

```
--log-driver=awslogs \
--log-opt awslogs-region=eu-north-1 \
--log-opt awslogs-group=/solar-app/container \
--log-opt awslogs-stream=solar-container \
```

8.2 View Logs

AWS Console → CloudWatch → Log groups → /solar-app/container → solar-container

8.3 Filter Logs

Filter	Shows
[PREDICTION]	Only prediction logs
ERROR	Only errors
mode=manual	Only manual predictions
mode=auto	Only auto predictions

8.4 Prediction Logging in utils.py

```
print(f"[PREDICTION] id={next_id} | datetime={datetime_str} | "
      f"irradiation={irradiation} | amb_temp={amb_temp} | "
      f"ac_power={round(ac_power, 2)} kW | mode={mode}")
```

9. Exam Day Checklist

9.1 Before Exam — Start Everything

Step	Action	Where
1	Start EC2 instance	AWS Console → EC2 → Start instance
2	Wait 2 minutes for EC2 to boot	—
3	Copy new Public IP	EC2 → Instances → Public IPv4 address
4	Update EC2_HOST secret	GitHub → Settings → Secrets → EC2_HOST
5	git push (if code changed)	Terminal → git push origin main
6	Wait for pipeline to complete	GitHub → Actions tab
7	Open Streamlit	http://NEW_IP:8501
8	Open FastAPI docs	http://NEW_IP:8000/docs

9.2 After Exam — Stop Everything

Step	Action	Cost Impact
1	Stop EC2 instance	Stops compute charges
2	Delete old ECR images (keep latest)	Saves storage cost
3	CloudWatch logs — do nothing	Free tier, no action needed
4	IAM role — do nothing	Always free

9.3 App URLs

Service	URL
Streamlit UI	http://EC2_PUBLIC_IP:8501
FastAPI Swagger	http://EC2_PUBLIC_IP:8000/docs
FastAPI Health	http://EC2_PUBLIC_IP:8000/health

10. AWS Costs Summary

Service	Running	Stopped	Notes
EC2 t3.micro	~$0.01/hr (~$7/mo)	$0	Stop when not in use
ECR Storage	~$0.03/mo	Same	302MB image, within free tier
CloudWatch	$0	$0	Within 5GB free tier
IAM Roles	$0	$0	Always free
EBS Storage	~$0.64/mo	Same	8GB volume, always charged

⚠ *Total cost when stopped: ~$0.67/month. Total when running: ~$8/month.*

11. Troubleshooting

Error	Cause	Fix
dial tcp:22 i/o timeout	EC2 stopped or port 22 blocked	Start EC2, open port 22 to 0.0.0.0/0
No such file: requirements.txt	Docker build from app/ but req in root	Add: cp requirements.txt app/requirements.txt
No space left on device	EC2 disk full with old images	Run: docker system prune -a -f on EC2
Model not loaded	model.pkl not in GitHub (gitignored)	git add app/model/model.pkl -f && git push
Cannot reach page	Wrong security group or EC2 stopped	Check launch-wizard-2 has ports 8000/8501 open
ECR repo not found	Wrong region in docker tag command	Use eu-north-1 in all commands
ModuleNotFoundError: streamlit	utils.py imports st but FastAPI doesn't have it	Replace st.warning with print() in utils.py

What you built

"I built an end-to-end MLOps pipeline for solar power generation forecasting. The system uses DVC for data versioning, MLflow for experiment tracking, and exposes the trained model through both a Streamlit UI and a FastAPI REST API. The entire application is containerized with Docker, automatically tested and deployed via GitHub Actions CI/CD to AWS EC2, with logs monitored in CloudWatch."

Why FastAPI alongside Streamlit

"Streamlit is for human users who interact via browser. FastAPI exposes the same model as a REST API so external systems, mobile apps, or automated scripts can get predictions programmatically without a UI. This makes the system production-ready."

Why DVC and MLflow are not in CI/CD

"DVC and MLflow are used locally for data versioning and experiment tracking during model development. Once the model is trained and exported as model.pkl, the CI/CD pipeline only needs to test and deploy the already-trained model. Running the full training pipeline in CI/CD would require remote data storage and MLflow server — unnecessary complexity for deployment automation."

Why Docker

"Docker ensures the app runs identically everywhere — on my laptop, in GitHub Actions, and on AWS EC2. It eliminates environment differences and makes deployment reproducible."

End of Document